

从 C++ 到计算系统：第一册实践卷

可执行文件、ABI、Linux 工具链与可信测量

CPU Performance Study

本卷是《从 C++ 到计算系统：第一册》的配套实践卷，专门承接习题、研究项目、完整代码、测试、benchmark 和机器证据报告。

前言

第一册正文讲清了 C++ 程序怎样经过编译、链接、装载、ABI 约定、指令执行和 Linux 工具链观察，最终成为一个正在运行的进程。但只读正文容易产生一种错觉：好像看懂概念就等于会做工程。真正的门槛在证据链里：你能不能拿一个自己编译出来的二进制，说明它是什么格式，入口在哪里，符号落在哪些 section，函数调用遵守什么 ABI，benchmark 结果是否被 checksum 保护，性能结论能否被反汇编、工具输出或测试复查。

本卷的贯穿项目叫 **Executable Evidence Kit**（可执行文件证据工具箱）。它不是一个玩具读文件程序，而是一条逐步扩展的研究项目：第一阶段读取字节并建立 checksum；第二阶段解析 ELF64 header；第三阶段解析 section table 和 string table；第四阶段解析 symbol table；第五阶段输出 report 和 CSV；第六阶段把 CLI、GTest/GMock、Google Benchmark、readelf/objdump/perf 观察和研究报告串起来。每一阶段都要回答同一个问题：这条结论来自哪里，怎样复查，当前工具还不能证明什么。

本卷补的是第一册最缺的两类训练：一类是章节习题，每章都要落到具体代码、命令和证据；另一类是研究项目，读者要从“照着跑”走到“能解释一个真实二进制”。因此本卷不会只列题目，也不会只贴模板。每个任务都先给问题背景，再说明最小实现、失败边界、测试要求和报告要求。

核心思想

第一册实践卷的目标不是把 ELF、ABI、汇编和 benchmark 各讲一次，而是让它们在同一个工程里互相校验：字节是事实，解析是解释，测试固定合同，工具输出交叉验证，报告只写证据能支撑的结论。

常见缩写和术语先导

本卷从第一章开始就会同时出现 C++、ELF、ABI、section、symbol、PMU、benchmark、GTest 等词。不要把它们当成孤立名词背下来；每个词都对应代码中的一个对象、命令行输出中的一个字段，或报告中的一条证据。

CPU: Central Processing Unit 中央处理器。它执行机器指令、访问寄存器和内存，并通过缓存、流水线、分支预测和乱序执行影响程序速度。

ELF: Executable and Linkable Format Linux 常见的可执行文件和目标文件格式。本卷只实现 ELF64 小端切片，重点训练 header、section、string table 和 symbol table 的证据链。

ABI: Application Binary Interface 应用二进制接口。它约定函数参数、返回值、寄存器保存、栈对齐、符号和链接边界，让不同编译单元能在二进制层面协作。

ISA: Instruction Set Architecture 指令集架构。x86-64、AArch64、RISC-V 都是 ISA；ELF header 的 machine 字段能告诉我们二进制面向哪类机器。

x86-64 64 位 x86 指令集。本卷第一版以 x86-64 为主要观察对象，但解析器会把 AArch64 和 RISC-V machine 字段也分类出来。

section ELF 文件中的一段逻辑区域，例如 `.text`、`.rodata`、`.symtab` 和 `.strtab`。section 是链接器和调试工具常用的组织方式。

symbol 符号。函数名、全局对象或链接器可见标记都可能成为符号。符号把源码名字、section、地址、大小和绑定关系连接起来。

string table 字符串表。ELF 中很多名字不直接写在结构体里，而是保存一个偏移，偏移指向某个字符串表。

entry address 入口地址。程序开始执行的虚拟地址，不等于 C++ 的 `main` 函数地址；运行时启动代码会先执行。

CLI: Command-Line Interface 命令行接口。本卷的 `cpulee_cli` 用文件路径和模式参数输出 report 或 section CSV。

CSV: Comma-Separated Values 逗号分隔表格。适合保存 section 摘要、benchmark 结果和可脚本复查的字段。

checksum 校验摘要。本卷使用它固定输入字节身份，避免报告只描述文件名而没有描述内容。

reference 参考实现。可以慢，但必须确定、清楚、可复查。实践中先有 reference，再让优化或工具扩展与它对齐。

oracle 判定器。它比较 reference 和待测实现，判断输出是否满足同一语义。

contract 合同。代码边界的明确约定：输入是什么，输出是什么，错误怎样表达，哪些行为暂不支持。

GTest: GoogleTest C++ 单元测试框架。本卷用它固定 ELF fixture、错误输入、符号上限和格式化输出。

GMock: GoogleMock GoogleTest 配套 mock 框架。复杂工程可用它替换外部依赖；本卷先通过 GTest/GMock 工程接入建立测试能力。

Google Benchmark C++ benchmark 库。本卷用它做解析路径 smoke，不用 benchmark 替代正确性测试。

PMU: Performance Monitoring Unit 性能监控单元。它统计 retired instructions、cache miss、branch miss 等硬件事件，是性能报告的重要证据来源。

perf Linux 性能分析工具，可读取 PMU 事件、采样热点和统计执行行为。没有 perf 权限时，报告要写明限制。

objdump 二进制工具，可反汇编、查看符号和 section 信息。它常用来把 C++ 函数和机器指令对应起来。

readelf ELF 检查工具，可查看 ELF header、section table、symbol table 等字段。它是本卷解析

器的交叉验证对象。

Debug/Release 常见构建类型。Debug 偏调试信息和可读性；Release 打开优化。两者的机器码和性能结论不能混用。

本卷结构

本卷只有两条主线。第一条是“合同主线”：从真实问题出发，说明为什么第一册需要一个可执行文件证据工具箱，以及每个阶段要固定哪些边界。第二条是“代码主线”：完整实现 [books/cpu-volume-1-practice/labs/executable_evidence](#)，并用测试、benchmark、CLI 和报告任务验收。

章节阅读顺序很简单：先读第一章，知道项目为什么这样设计；再读第二章，按阶段写代码、跑测试和做研究报告；最后读源码清单，把书中片段和完整工程对上。

目录

常见缩写和术语先导	iii
本卷结构	v
第一部分 从源码到二进制证据	
第 1 章 Executable Evidence Kit: 从问题推出工程边界	2
第二部分 Executable Evidence Kit 实践	
第 2 章 代码实践卷: Executable Evidence Kit	6
完整源码清单	9
术语表	31

第零部分

从源码到二进制证据

第 1 章

Executable Evidence Kit: 从问题推出工程边界

第一册正文里有一个反复出现的问题：我们说“C++ 代码编译成机器指令”“函数遵守 ABI”“benchmark 要可信”，但读者真正动手时，常常不知道从哪里开始验证。最糟的学习路径是直接背结论：**.text** 放代码，System V ABI 用寄存器传参，Release 比 Debug 快，perf 能看性能。结论没有错，但没有证据链时，读者只会复述名词。

本章从一个具体任务开始：给定一个本地可执行文件，写一个 C++20 工具输出它的基本证据。第一版只支持 ELF64 little-endian，也就是 Linux 上常见的 64 位小端 ELF 文件。输入是一串字节；输出是一份 report，包括 magic 是否正确、是否 ELF64、是否小端、machine 字段、entry address、section 列表、symbol 列表、checksum 和 diagnostics。这个任务足够小，能完整实现；也足够贴近第一册，能自然牵出编译、链接、装载、ABI、汇编和测量。

1.1 一、从一个最小问题开始

设想你编译了一个程序：

```
1 g++ -O2 -g hello.cpp -o hello
```

你想回答六个问题：

1. 它到底是不是 ELF 文件？
2. 它面向哪种机器架构？
3. 程序入口地址是多少？
4. 哪些 section 存在，**.text** 在文件中从哪里开始，占多少字节？
5. 符号表里有没有你写的函数名，它属于哪个 section？
6. 这份报告对应的是哪个具体文件内容，而不是同名旧文件？

最直接的做法是用 **readelf-h**、**readelf-S**、**readelf-s** 和 **objdump-d**。这些工具很强，但如果读者从工具输出开始，容易只会看结果，不知道字段为什么这样组织。本卷让你先实现一个小切片，再拿成熟工具交叉验证。实现不是为了替代 readelf，而是为了让每个字段都经过你自己的手。

核心思想

实践卷的第一条规则：工具输出不是魔法。能自己解析一小段格式，才知道成熟工具在帮你省掉哪些细节，也知道它的输出不应该被盲信。

1.2 二、为什么先做 ELF，而不是先做 benchmark

第一册有性能测量内容，为什么新实践卷不直接扩展 benchmark？原因是 benchmark 也需要二进制证据。你说一个循环更快，至少要知道当前跑的是 Debug 还是 Release，热函数是否还存在，符号是否被 inline 或 strip，**.text** 是否变化，反汇编里有没有意外调用。若不理解二进制结构，benchmark 报告很容易只剩时间数字。

Executable Evidence Kit 先建立“文件事实”：字节、checksum、header、section、symbol。然后再进入“运行事实”：命令行参数、输入规模、计时边界、perf 事件、反汇编片段。两类事实要互相支撑。比如同一个源码函数在 Debug 中保留独立符号，在 Release 中可能被 inline；这不是“工具坏了”，而是优化改变了机器证据的形态。

常见误区

不要把文件名当作证据。报告里写 `hello` 没有意义，因为同名文件可能被重编译覆盖。至少要保存 `byte size` 和 `checksum`，必要时保存构建命令、编译器版本和 `git commit`。

1.3 三、合同：第一版支持什么，不支持什么

第一版合同要窄，但必须清楚。本卷实现的解析器只承诺：

- 1. 输入是内存中的字节序列，不直接绑定文件系统；
- 2. 支持 ELF magic 检查；
- 3. 支持 ELF64 little-endian；
- 4. 读取 object type、machine、entry address；
- 5. 读取 section header table 和 section-name string table；
- 6. 读取 `SHT_SYMTAB` 与 `SHT_DYNSYM` 的符号摘要；
- 7. 输出稳定文本 report 和 section CSV；
- 8. 对不支持或越界的输入给 diagnostics，而不是假装成功。

它暂不承诺解析 program header、relocation、DWARF 调试信息、动态链接器状态、PIE 装载后的真实运行地址，也不承诺支持 32 位、大端或非 ELF 格式。限制不是缺陷，限制写清楚就是合同的一部分。

1.4 四、从特例推到规律

先看最小特例。一个合格 ELF 文件开头四个字节必须是 `0x7F'E''L''F'`。如果这四个字节不对，后面任何字段都不能读，因为你不知道它是什么格式。第二个特例是文件只有 magic，却不足 64 字节。magic 正确只能说明“像 ELF”，不能说明 ELF64 header 完整；此时继续读 entry address 就是在越界解释垃圾。

由这两个特例可以推出第一条规律：每读一个结构前，先证明字节范围存在。代码里的 `has_range` 就是这个规律。再看 section table：header 会告诉我们 section header offset、entry size、count 和 section-name string table index。若 entry size 小于 ELF64 section header，说明当前解析器不能安全读取；若 string table index 越界，就不能把 name offset 翻译成名字。

符号表也是同一套规律。symbol 的 name 字段不是字符串，而是字符串表偏移；symbol 的 section index 也不是名字，而是 section table 下标。因此解析顺序不能乱：先 header，再 section table，再 section-name string table，再 symbol table 和 symbol string table。这个顺序就是工程结构，不是格式细节。

心智模型

ELF 解析像读一张地图的目录。header 告诉你目录在哪里，section table 告诉你每个区域的偏移和大小，string table 把数字偏移翻译成名字，symbol table 再把函数名、section 和地址连起来。

1.5 五、验收：代码、测试、工具、报告四件事一起过

本卷不接受“我写了一个函数但没跑”的实践。每个阶段都有四类证据：

证据	形式	通过标准
代码	C++20 library 与 CLI	接口清楚，输入和错误路径不混在 main 里
测试	GTest/GMock	synthetic ELF、坏 magic、短 header、符号上限都有断言
工具	readelf/objdump/perf 或替代命令	至少交叉验证 header、section、symbol 中的三个字段
报告	Markdown 或 CSV	写清构建命令、输入 checksum、结论和限制

最终研究项目不是把本卷代码抄一遍，而是选择一个自己编译的小程序，生成二进制证据报告。报告必须把第一册正文里的概念落到同一个对象上：源码函数对应哪个 symbol，symbol 落在哪个 section，section 在文件里的偏移是多少，反汇编能否看到函数机器码，benchmark 结论是否和当前构建产物一致。

第零部分

Executable Evidence Kit 实践

第 2 章

代码实践卷：Executable Evidence Kit

本章对应仓库目录：

```
1 books/cpu-volume-1-practice/labs/executable_evidence
```

这不是附带样例，而是第一册的主线实践工程。读者应该把它当成一条从零到研究报告的流水线：先让代码能解析一个人工构造的最小 ELF；再让它检查真实二进制；然后把 readelf、objdump、perf 和 benchmark 接进报告。每一步都保留测试和证据，不靠口头解释。

2.1 一、构建、运行和目录结构

构建命令如下：

```
1 cmake -S books/cpu-volume-1-practice \
2     -B books/cpu-volume-1-practice/build/executable-evidence-debug \
3     -DCMAKE_BUILD_TYPE=Debug
4 cmake --build books/cpu-volume-1-practice/build/executable-evidence-debug
5 ctest --test-dir books/cpu-volume-1-practice/build/executable-evidence-debug \
6     --output-on-failure
```

CLI 可以检查自己：

```
1 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
   ↵ executable_evidence/cpu1ee_cli \
2 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
   ↵ executable_evidence/cpu1ee_cli
```

目录结构按责任分层：

路径	责任
<code>include/cpu1/executable_evidence.hpp</code>	对外合同：输入配置、section 摘要、symbol 摘要、report 和格式化函数
<code>include/cpu1/elf_fixture.hpp</code>	synthetic ELF 构造器，供测试和 benchmark 共享
<code>src/executable_evidence.cpp</code>	ELF64 little-endian 解析、checksum、report 和 CSV 输出
<code>src/main.cpp</code>	CLI 入口，只负责参数、文件读取和输出模式
<code>tests/executable_evidence_tests.cpp</code>	GTest/GMock 测试，覆盖正常 fixture 和错误边界
<code>bench/executable_evidence_bench.cpp</code>	Google Benchmark smoke，只测解析路径

2.2 二、分阶段实现和习题

阶段 0：字节和 checksum。 第一阶段不要急着解释 ELF。先把文件读成 `std::vector<std::uint8_t>`，用 `checksum_bytes` 得到稳定摘要。习题是构造两个只差一个字节的输入，证明 checksum 不同；再用同一输入重复计算，证明 checksum 稳定。

阶段 1: ELF magic 和 header。 先只检查前四个 magic 字节, 再检查输入至少有 ELF64 header 长度, 最后读取 class、endianness、object type、machine 和 entry address。习题是写三个坏输入: 完全不是 ELF、只有 magic 但太短、ELF32 class。每个坏输入都必须得到明确 diagnostic。

阶段 2: section table。 header 给出 section header offset、entry size、count 和 section-name string table index。实现时先读 raw section, 再用 `.shstrtab` 把 name offset 翻译成名字。习题是解释为什么 section 名字不能在读 raw section 时立刻解析: 因为当时还不知道 string table 的字节范围是否安全。

阶段 3: symbol table。 符号表依赖两个事实: symbol table 自己的位置和大小, 以及它 link 到的字符串表。习题是找到 `_start` 的 symbol, 说明 binding、type、section index、value 和 size 各自表达什么。不要把 value 直接叫“文件偏移”, 它通常是虚拟地址或链接视角的地址。

阶段 4: CLI 和 CSV。 CLI 的职责是薄的: 读取文件, 调用 library, 选择 report 或 CSV。习题是用 `--sections-csv` 输出 section 表, 再写脚本检查表头是否固定为 `name,type,flags,address,offset,size`。

阶段 5: benchmark smoke。 Google Benchmark 只测 synthetic ELF 的解析路径, 不测文件读取和打印。习题是解释为什么输入构造必须放在计时循环外: 否则测到的是构造成本、分配成本和缓存状态, 不是解析器本身。

2.3 三、每章实践任务矩阵

第一册章节	实践任务	验收证据
心智模型	画出 source、compiler、ELF、loader、process 的证据链	报告能区分文件事实和运行事实
程序到进程	用 CLI 检查一个自己编译的小程序	保存 byte size、checksum、machine、entry
编译器 IR 与优化	分别检查 Debug 和 Release 产物	说明 section、symbol 或 size 差异
CPU 执行指令	对一个函数用 <code>objdump</code> 找机器码	能指出函数符号和 <code>.text</code> 的关系
数据、内存、指针、布局	解释 section offset、size 和字节范围检查	不越界读取, 不把虚拟地址当文件偏移
Linux 工具与调试	用 <code>readelf</code> 交叉验证三个字段	字段名、数值和工具命令写入报告
汇编语法	对照反汇编中的操作数和地址	能说明至少一条指令访问寄存器还是内存
整数指令	检查解析器的 little-endian 读取函数	有 u16/u32/u64 小端样例测试
控制流	解释错误输入为什么提前返回	diagnostics 覆盖 magic、短 header、不支持 class
System V ABI	找一个未被 inline 的函数符号	说明函数名、binding、type、section index
C++ 与汇编互操作	比较 C++ 名字、符号名字和工具输出	写清 name mangling 或优化影响
测量基础	运行 benchmark smoke	说明只测解析, 不测 I/O 和打印
benchmark 设计	写一份坏测量分析	指出计时边界里混入的无关成本

2.4 四、递进研究项目

研究项目分六次提交, 不要求一次写完。

阶段	交付物	通过标准
0	synthetic ELF 测试	正常 fixture 能解析出 <code>.text</code> 和 <code>_start</code>
1	错误输入测试	非 ELF、短 header、不支持 class 都有 diagnostic

2	真实二进制 report	对一个自己编译的程序输出 checksum、machine、entry、sections
3	工具交叉验证	readelf 或 objdump 至少验证三个字段
4	符号与 ABI 观察	选择一个函数解释 symbol、section、地址和优化影响
5	benchmark 与测量限制	benchmark smoke 通过, 报告说明未测 I/O、打印和真实 workload
6	最终研究报告	结论只来自测试、CLI 输出、工具输出和明确限制

最终报告的核心不是“我看到了很多字段”，而是“这些字段怎样互相证明”。例如：`cpulee_cli` 输出 machine 为 x86-64，`readelf` header 同样显示 x86-64；`.text` 的 offset 和 size 能解释 `objdump` 反汇编来自哪里；symbol table 中的函数名能和源码函数对应；checksum 证明报告绑定的是这一次构建产物。

完整源码清单

本附录从 [books/cpu-volume-1-practice/labs/executable_evidence](#) 直接读入完整工程源码。读者不要只看正文里的片段；真正的能力来自完整构建、完整测试和完整报告。

一、构建入口

Listing 2.1 第一册实践卷顶层 CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume1Practice LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 option(CPU_VOLUME_1_PRACTICE_BUILD_LABS "Build CPU Volume 1 practice labs" ON)
10
11 if(CPU_VOLUME_1_PRACTICE_BUILD_LABS)
12     enable_testing()
13     add_subdirectory(labs/executable_evidence)
14 endif()
```

Listing 2.2 Executable Evidence Kit README.md

```
1 # Executable Evidence Kit
2
3 这是第一册实践卷的贯穿项目。它把“C++ 源码如何成为进程”落成一个可测试工具：读取 ↵
    ↵ ELF64 小端可执行文件，解析 header、section table、string table 和 symbol ↵
    ↵ table，输出稳定文本报告和 section CSV。
4
5 ## 阶段路线
6
7 1. 字节读取：用 `read_binary_file` 把文件内容读成 `std::vector<std::uint8_t>`，↵
    ↵ 并用 checksum 固定输入身份。
8 2. ELF header：检查 magic、class、endianness、machine 和 entry address，区分“不↵
    ↵ 是 ELF”“不是 ELF64”“不是小端”。
9 3. Section table：解析 `.text`、`.shstrtab`、`.symtab` 和 `.strtab`，把 section↵
    ↵ name、offset、size 和 flags 输出成 CSV。
10 4. Symbol table：解析符号名、绑定、类型、所在 section、地址和大小，把 ABI/链接↵
    ↵ 视角接回源码函数。
11 5. CLI 证据：让工具可以检查自己或其他二进制，输出 report 或 section CSV。
12 6. 测试与 benchmark：用 GTest/GMock 固定边界，用 Google Benchmark 对解析路径做 ↵
    ↵ smoke。
13
14 ## 构建
15
16 ```bash
```

```

17 cmake -S books/cpu-volume-1-practice \
18     -B books/cpu-volume-1-practice/build/executable-evidence-debug \
19     -DCMAKE_BUILD_TYPE=Debug
20 cmake --build books/cpu-volume-1-practice/build/executable-evidence-debug
21 ctest --test-dir books/cpu-volume-1-practice/build/executable-evidence-debug \
22     --output-on-failure
23 ```
24
25 ## 运行
26
27 ```bash
28 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
29     ↵ executable_evidence/cpu1ee_cli \
30 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
31     ↵ executable_evidence/cpu1ee_cli
32 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
33     ↵ executable_evidence/cpu1ee_cli \
34     --sections-csv
35 ```
36
37 ## 研究项目
38
39 最终项目不是“看懂 ELF 文件格式”这么窄，而是提交一份二进制证据报告：
40 - 选一个自己编译的小程序；
41 - 用 `cpu1ee_cli` 记录 header、sections、symbols 和 checksum；
42 - 用 `readelf` 或 `objdump` 交叉验证至少三个字段；
43 - 对一个 C++ 函数解释它的符号、section、入口地址或机器码位置；
44 - 说明 Debug/Release、优化等级或 strip 对报告有什么影响；
45 - 把不可验证的结论写成限制，不写成事实。

```

Listing 2.3 Executable Evidence Kit CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.31)
2 project(CPU1ExecutableEvidence LANGUAGES CXX)
3
4 list(APPEND CMAKE_MODULE_PATH "${CMAKE_CURRENT_LIST_DIR}/../../../../../cmake")
5 include(BookLabDependencies)
6 book_labs_enable_gtest()
7 book_labs_enable_benchmark()
8
9 add_library(cpu1_executable_evidence STATIC
10     src/executable_evidence.cpp
11 )
12
13 target_include_directories(cpu1_executable_evidence
14     PUBLIC
15     ${CMAKE_CURRENT_SOURCE_DIR}/include
16 )

```

```

17
18 target_compile_features(cpu1_executable_evidence PUBLIC cxx_std_20)
19
20 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
21     target_compile_options(cpu1_executable_evidence PRIVATE -Wall -Wextra -Wpedantic)
22 endif()
23
24 add_executable(cpu1ee_cli
25     src/main.cpp
26 )
27
28 target_link_libraries(cpu1ee_cli PRIVATE cpu1_executable_evidence)
29
30 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
31     target_compile_options(cpu1ee_cli PRIVATE -Wall -Wextra -Wpedantic)
32 endif()
33
34 add_executable(cpu1ee_tests
35     tests/executable_evidence_tests.cpp
36 )
37
38 target_link_libraries(cpu1ee_tests
39     PRIVATE
40         cpu1_executable_evidence
41         GTest::gmock_main
42 )
43
44 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
45     target_compile_options(cpu1ee_tests PRIVATE -Wall -Wextra -Wpedantic)
46 endif()
47
48 add_test(NAME cpu1ee_tests COMMAND ${CMAKE_TARGET_FILE:cpu1ee_tests})
49
50 add_executable(cpu1ee_bench
51     bench/executable_evidence_bench.cpp
52 )
53
54 target_link_libraries(cpu1ee_bench
55     PRIVATE
56         cpu1_executable_evidence
57         benchmark::benchmark
58 )
59
60 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
61     target_compile_options(cpu1ee_bench PRIVATE -Wall -Wextra -Wpedantic)
62 endif()

```

二、公共合同

Listing 2.4 include/cpu1/executable_evidence.hpp

```
1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <span>
6 #include <string>
7 #include <string_view>
8 #include <vector>
9
10 namespace cpu1 {
11
12 struct InspectionConfig {
13     bool include_symbols{true};
14     std::uint64_t max_symbols{128};
15 };
16
17 struct SectionSummary {
18     std::string name;
19     std::uint32_t type{};
20     std::uint64_t flags{};
21     std::uint64_t address{};
22     std::uint64_t offset{};
23     std::uint64_t size{};
24 };
25
26 struct SymbolSummary {
27     std::string name;
28     std::string section_name;
29     std::uint8_t type{};
30     std::uint8_t binding{};
31     std::uint16_t section_index{};
32     std::uint64_t value{};
33     std::uint64_t size{};
34 };
35
36 struct ElfReport {
37     std::string source_name;
38     bool valid_magic{};
39     bool is_elf64{};
40     bool is_little_endian{};
41     std::uint16_t object_type{};
42     std::uint16_t machine{};
43     std::uint64_t entry_address{};
44     std::uint64_t checksum{};
45     std::uint64_t byte_size{};
46     std::vector<SectionSummary> sections;
47     std::vector<SymbolSummary> symbols;
48     std::vector<std::string> diagnostics;
49 };
50
51 std::vector<std::uint8_t> read_binary_file(const std::filesystem::path& path);
```

```

52 std::uint64_t checksum_bytes(std::span<const std::uint8_t> bytes);
53 std::string classify_machine(std::uint16_t machine);
54 ElfReport inspect_elf(
55     std::span<const std::uint8_t> bytes,
56     std::string_view source_name,
57     const InspectionConfig& config = {});
58 std::string format_report(const ElfReport& report);
59 std::string format_sections_csv(const ElfReport& report);
60
61 } // namespace cpu1

```

Listing 2.5 include/cpu1/elf_fixture.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <cstdint>
5 #include <string_view>
6 #include <vector>
7
8 namespace cpu1 {
9
10 constexpr std::size_t FIXTURE_ELF_SIZE = 704;
11 constexpr std::size_t FIXTURE_ELF_IDENT_SIZE = 16;
12 constexpr std::size_t FIXTURE_ELF_HEADER_SIZE = 64;
13 constexpr std::size_t FIXTURE_ELF_SH_OFFSET = 384;
14 constexpr std::size_t FIXTURE_ELF_SH_ENTRY_SIZE = 64;
15 constexpr std::size_t FIXTURE_ELF_SH_COUNT = 5;
16 constexpr std::size_t FIXTURE_ELF_SHSTR_INDEX = 2;
17 constexpr std::size_t FIXTURE_ELF_TEXT_OFFSET = 96;
18 constexpr std::size_t FIXTURE_ELF_TEXT_SIZE = 4;
19 constexpr std::size_t FIXTURE_ELF_SHSTRTAB_OFFSET = 128;
20 constexpr std::size_t FIXTURE_ELF_STRTAB_OFFSET = 176;
21 constexpr std::size_t FIXTURE_ELF_SYMTAB_OFFSET = 208;
22 constexpr std::size_t FIXTURE_ELF_SYMTAB_ENTRY_SIZE = 24;
23 constexpr std::uint64_t FIXTURE_ELF_ENTRY = 0x401000;
24 constexpr std::uint16_t FIXTURE_ELF_EXEC_TYPE = 2;
25 constexpr std::uint16_t FIXTURE_ELF_X86_64 = 62;
26 constexpr std::uint32_t FIXTURE_ELF_VERSION = 1;
27 constexpr std::uint32_t FIXTURE_ELF_PROGBITS = 1;
28 constexpr std::uint32_t FIXTURE_ELF_SYMTAB = 2;
29 constexpr std::uint32_t FIXTURE_ELF_STRTAB = 3;
30 constexpr std::uint64_t FIXTURE_ELF_ALLOC_EXEC_FLAGS = 6;
31 constexpr std::uint8_t FIXTURE_ELF64_CLASS = 2;
32 constexpr std::uint8_t FIXTURE_ELF_LITTLE_ENDIAN = 1;
33 constexpr std::uint8_t FIXTURE_ELF_MAGIC_0 = 0x7F;
34 constexpr std::uint8_t FIXTURE_ELF_MAGIC_1 = 'E';
35 constexpr std::uint8_t FIXTURE_ELF_MAGIC_2 = 'L';
36 constexpr std::uint8_t FIXTURE_ELF_MAGIC_3 = 'F';
37 constexpr std::uint8_t FIXTURE_SYMBOL_GLOBAL_FUNC = 0x12;
38 constexpr std::uint8_t FIXTURE_SYMBOL_LOCAL_FUNC = 0x02;
39 constexpr std::uint8_t FIXTURE_TEXT_BYTE_0 = 0x55;

```

```

40 constexpr std::uint8_t FIXTURE_TEXT_BYTE_1 = 0x48;
41 constexpr std::uint8_t FIXTURE_TEXT_BYTE_2 = 0x89;
42 constexpr std::uint8_t FIXTURE_TEXT_BYTE_3 = 0xE5;
43 constexpr std::size_t FIXTURE_ELF_TYPE_OFFSET = 16;
44 constexpr std::size_t FIXTURE_ELF_MACHINE_OFFSET = 18;
45 constexpr std::size_t FIXTURE_ELF_VERSION_OFFSET = 20;
46 constexpr std::size_t FIXTURE_ELF_ENTRY_OFFSET = 24;
47 constexpr std::size_t FIXTURE_ELF_SECTION_HEADER_OFFSET = 40;
48 constexpr std::size_t FIXTURE_ELF_HEADER_SIZE_OFFSET = 52;
49 constexpr std::size_t FIXTURE_ELF_SECTION_ENTRY_SIZE_OFFSET = 58;
50 constexpr std::size_t FIXTURE_ELF_SECTION_COUNT_OFFSET = 60;
51 constexpr std::size_t FIXTURE_ELF_SECTION_NAME_INDEX_OFFSET = 62;
52 constexpr std::size_t FIXTURE_SECTION_NAME_OFFSET = 0;
53 constexpr std::size_t FIXTURE_SECTION_TYPE_OFFSET = 4;
54 constexpr std::size_t FIXTURE_SECTION_FLAGS_OFFSET = 8;
55 constexpr std::size_t FIXTURE_SECTION_ADDRESS_OFFSET = 16;
56 constexpr std::size_t FIXTURE_SECTION_FILE_OFFSET = 24;
57 constexpr std::size_t FIXTURE_SECTION_SIZE_OFFSET = 32;
58 constexpr std::size_t FIXTURE_SECTION_LINK_OFFSET = 40;
59 constexpr std::size_t FIXTURE_SECTION_ENTRY_SIZE_OFFSET = 56;
60 constexpr std::size_t FIXTURE_SYMBOL_NAME_OFFSET = 0;
61 constexpr std::size_t FIXTURE_SYMBOL_INFO_OFFSET = 4;
62 constexpr std::size_t FIXTURE_SYMBOL_SECTION_INDEX_OFFSET = 6;
63 constexpr std::size_t FIXTURE_SYMBOL_VALUE_OFFSET = 8;
64 constexpr std::size_t FIXTURE_SYMBOL_SIZE_OFFSET = 16;
65
66 inline void put_u16(std::vector<std::uint8_t>& bytes, std::size_t offset, std::←
    ↪ uint16_t value)
67 {
68     bytes[offset] = static_cast<std::uint8_t>(value & 0xFFU);
69     bytes[offset + 1] = static_cast<std::uint8_t>((value >> 8U) & 0xFFU);
70 }
71
72 inline void put_u32(std::vector<std::uint8_t>& bytes, std::size_t offset, std::←
    ↪ uint32_t value)
73 {
74     constexpr std::size_t BYTE_BITS = 8;
75     for (std::size_t i = 0; i < sizeof(std::uint32_t); ++i) {
76         bytes[offset + i] = static_cast<std::uint8_t>((value >> (i * BYTE_BITS)←
    ↪ ) & 0xFFU);
77     }
78 }
79
80 inline void put_u64(std::vector<std::uint8_t>& bytes, std::size_t offset, std::←
    ↪ uint64_t value)
81 {
82     constexpr std::size_t BYTE_BITS = 8;
83     for (std::size_t i = 0; i < sizeof(std::uint64_t); ++i) {
84         bytes[offset + i] = static_cast<std::uint8_t>((value >> (i * BYTE_BITS)←
    ↪ ) & 0xFFU);
85     }
86 }

```

```

87
88 inline void put_string(std::vector<std::uint8_t>& bytes, std::size_t offset, ↵
    ↵ std::string_view value)
89 {
90     for (std::size_t i = 0; i < value.size(); ++i) {
91         bytes[offset + i] = static_cast<std::uint8_t>(value[i]);
92     }
93 }
94
95 inline void put_section_header(
96     std::vector<std::uint8_t>& bytes,
97     std::size_t index,
98     std::uint32_t name_offset,
99     std::uint32_t type,
100     std::uint64_t flags,
101     std::uint64_t address,
102     std::uint64_t file_offset,
103     std::uint64_t size,
104     std::uint32_t link,
105     std::uint64_t entry_size)
106 {
107     const std::size_t offset = FIXTURE_ELF_SH_OFFSET + index * ↵
    ↵ FIXTURE_ELF_SH_ENTRY_SIZE;
108     put_u32(bytes, offset + FIXTURE_SECTION_NAME_OFFSET, name_offset);
109     put_u32(bytes, offset + FIXTURE_SECTION_TYPE_OFFSET, type);
110     put_u64(bytes, offset + FIXTURE_SECTION_FLAGS_OFFSET, flags);
111     put_u64(bytes, offset + FIXTURE_SECTION_ADDRESS_OFFSET, address);
112     put_u64(bytes, offset + FIXTURE_SECTION_FILE_OFFSET, file_offset);
113     put_u64(bytes, offset + FIXTURE_SECTION_SIZE_OFFSET, size);
114     put_u32(bytes, offset + FIXTURE_SECTION_LINK_OFFSET, link);
115     put_u64(bytes, offset + FIXTURE_SECTION_ENTRY_SIZE_OFFSET, entry_size);
116 }
117
118 inline void put_symbol(
119     std::vector<std::uint8_t>& bytes,
120     std::size_t index,
121     std::uint32_t name_offset,
122     std::uint8_t info,
123     std::uint16_t section_index,
124     std::uint64_t value,
125     std::uint64_t size)
126 {
127     const std::size_t offset = FIXTURE_ELF_SYMTAB_OFFSET + index * ↵
    ↵ FIXTURE_ELF_SYMTAB_ENTRY_SIZE;
128     put_u32(bytes, offset + FIXTURE_SYMBOL_NAME_OFFSET, name_offset);
129     bytes[offset + FIXTURE_SYMBOL_INFO_OFFSET] = info;
130     put_u16(bytes, offset + FIXTURE_SYMBOL_SECTION_INDEX_OFFSET, section_index)↵
    ↵ ;
131     put_u64(bytes, offset + FIXTURE_SYMBOL_VALUE_OFFSET, value);
132     put_u64(bytes, offset + FIXTURE_SYMBOL_SIZE_OFFSET, size);
133 }
134

```

```

135 inline std::vector<std::uint8_t> make_synthetic_elf()
136 {
137     std::vector<std::uint8_t> bytes(FIXTURE_ELF_SIZE, 0);
138     bytes[0] = FIXTURE_ELF_MAGIC_0;
139     bytes[1] = FIXTURE_ELF_MAGIC_1;
140     bytes[2] = FIXTURE_ELF_MAGIC_2;
141     bytes[3] = FIXTURE_ELF_MAGIC_3;
142     bytes[4] = FIXTURE_ELF64_CLASS;
143     bytes[5] = FIXTURE_ELF_LITTLE_ENDIAN;
144     bytes[6] = 1;
145
146     put_u16(bytes, FIXTURE_ELF_TYPE_OFFSET, FIXTURE_ELF_EXEC_TYPE);
147     put_u16(bytes, FIXTURE_ELF_MACHINE_OFFSET, FIXTURE_ELF_X86_64);
148     put_u32(bytes, FIXTURE_ELF_VERSION_OFFSET, FIXTURE_ELF_VERSION);
149     put_u64(bytes, FIXTURE_ELF_ENTRY_OFFSET, FIXTURE_ELF_ENTRY);
150     put_u64(bytes, FIXTURE_ELF_SECTION_HEADER_OFFSET, FIXTURE_ELF_SH_OFFSET);
151     put_u16(bytes, FIXTURE_ELF_HEADER_SIZE_OFFSET, FIXTURE_ELF_HEADER_SIZE);
152     put_u16(bytes, FIXTURE_ELF_SECTION_ENTRY_SIZE_OFFSET, ←
    ↪ FIXTURE_ELF_SH_ENTRY_SIZE);
153     put_u16(bytes, FIXTURE_ELF_SECTION_COUNT_OFFSET, FIXTURE_ELF_SH_COUNT);
154     put_u16(bytes, FIXTURE_ELF_SECTION_NAME_INDEX_OFFSET, ←
    ↪ FIXTURE_ELF_SHSTR_INDEX);
155
156     bytes[FIXTURE_ELF_TEXT_OFFSET + 0] = FIXTURE_TEXT_BYTE_0;
157     bytes[FIXTURE_ELF_TEXT_OFFSET + 1] = FIXTURE_TEXT_BYTE_1;
158     bytes[FIXTURE_ELF_TEXT_OFFSET + 2] = FIXTURE_TEXT_BYTE_2;
159     bytes[FIXTURE_ELF_TEXT_OFFSET + 3] = FIXTURE_TEXT_BYTE_3;
160
161     put_string(bytes, FIXTURE_ELF_SHSTRTAB_OFFSET + 1, ".text");
162     put_string(bytes, FIXTURE_ELF_SHSTRTAB_OFFSET + 7, ".shstrtab");
163     put_string(bytes, FIXTURE_ELF_SHSTRTAB_OFFSET + 17, ".symtab");
164     put_string(bytes, FIXTURE_ELF_SHSTRTAB_OFFSET + 25, ".strtab");
165     put_string(bytes, FIXTURE_ELF_STRTAB_OFFSET + 1, "_start");
166     put_string(bytes, FIXTURE_ELF_STRTAB_OFFSET + 8, "helper");
167
168     put_symbol(bytes, 1, 1, FIXTURE_SYMBOL_GLOBAL_FUNC, 1, FIXTURE_ELF_ENTRY,
169                 FIXTURE_ELF_TEXT_SIZE);
170     put_symbol(bytes, 2, 8, FIXTURE_SYMBOL_LOCAL_FUNC, 1,
171                 FIXTURE_ELF_ENTRY + FIXTURE_ELF_TEXT_SIZE, 8);
172
173     put_section_header(bytes, 1, 1, FIXTURE_ELF_PROGBITS, ←
    ↪ FIXTURE_ELF_ALLOC_EXEC_FLAGS,
174                        FIXTURE_ELF_ENTRY, FIXTURE_ELF_TEXT_OFFSET, ←
    ↪ FIXTURE_ELF_TEXT_SIZE, 0, 0);
175     put_section_header(bytes, 2, 7, FIXTURE_ELF_STRTAB, 0, 0, ←
    ↪ FIXTURE_ELF_SHSTRTAB_OFFSET, 33, 0,
176                        0);
177     put_section_header(bytes, 3, 17, FIXTURE_ELF_SYMTAB, 0, 0, ←
    ↪ FIXTURE_ELF_SYMTAB_OFFSET,
178                        FIXTURE_ELF_SYMTAB_ENTRY_SIZE * 3, 4, ←
    ↪ FIXTURE_ELF_SYMTAB_ENTRY_SIZE);
179     put_section_header(bytes, 4, 25, FIXTURE_ELF_STRTAB, 0, 0, ←

```

```

    ↪ FIXTURE_ELF_STRTAB_OFFSET, 15, 0,
180         0);
181
182     return bytes;
183 }
184
185 } // namespace cpu1

```

三、实现和 CLI

Listing 2.6 src/executable_evidence.cpp

```

1 #include <cpu1/executable_evidence.hpp>
2
3 #include <algorithm>
4 #include <cstdint>
5 #include <fstream>
6 #include <iomanip>
7 #include <sstream>
8 #include <stdexcept>
9
10 namespace cpu1 {
11 namespace {
12
13 constexpr std::uint8_t ELF_MAGIC_0 = 0x7F;
14 constexpr std::uint8_t ELF_MAGIC_1 = 'E';
15 constexpr std::uint8_t ELF_MAGIC_2 = 'L';
16 constexpr std::uint8_t ELF_MAGIC_3 = 'F';
17 constexpr std::size_t ELF_IDENT_SIZE = 16;
18 constexpr std::size_t ELF_IDENT_CLASS_OFFSET = 4;
19 constexpr std::size_t ELF_IDENT_DATA_OFFSET = 5;
20 constexpr std::uint8_t ELF_CLASS_64 = 2;
21 constexpr std::uint8_t ELF_DATA_LITTLE_ENDIAN = 1;
22
23 constexpr std::size_t ELF64_HEADER_MIN_SIZE = 64;
24 constexpr std::size_t ELF64_TYPE_OFFSET = 16;
25 constexpr std::size_t ELF64_MACHINE_OFFSET = 18;
26 constexpr std::size_t ELF64_ENTRY_OFFSET = 24;
27 constexpr std::size_t ELF64_SECTION_HEADER_OFFSET = 40;
28 constexpr std::size_t ELF64_HEADER_SECTION_ENTRY_SIZE_OFFSET = 58;
29 constexpr std::size_t ELF64_SECTION_COUNT_OFFSET = 60;
30 constexpr std::size_t ELF64_SECTION_NAME_INDEX_OFFSET = 62;
31
32 constexpr std::size_t ELF64_SECTION_NAME_OFFSET = 0;
33 constexpr std::size_t ELF64_SECTION_TYPE_OFFSET = 4;
34 constexpr std::size_t ELF64_SECTION_FLAGS_OFFSET = 8;
35 constexpr std::size_t ELF64_SECTION_ADDRESS_OFFSET = 16;
36 constexpr std::size_t ELF64_SECTION_FILE_OFFSET = 24;
37 constexpr std::size_t ELF64_SECTION_SIZE_OFFSET = 32;
38 constexpr std::size_t ELF64_SECTION_LINK_OFFSET = 40;
39 constexpr std::size_t ELF64_SECTION_ENTRY_SIZE_OFFSET = 56;

```

```

40 constexpr std::size_t ELF64_SECTION_HEADER_SIZE = 64;
41
42 constexpr std::size_t ELF64_SYMBOL_NAME_OFFSET = 0;
43 constexpr std::size_t ELF64_SYMBOL_INFO_OFFSET = 4;
44 constexpr std::size_t ELF64_SYMBOL_SECTION_INDEX_OFFSET = 6;
45 constexpr std::size_t ELF64_SYMBOL_VALUE_OFFSET = 8;
46 constexpr std::size_t ELF64_SYMBOL_SIZE_OFFSET = 16;
47 constexpr std::size_t ELF64_SYMBOL_ENTRY_SIZE = 24;
48
49 constexpr std::uint32_t ELF_SECTION_TYPE_SYMTAB = 2;
50 constexpr std::uint32_t ELF_SECTION_TYPE_DYNSYM = 11;
51 constexpr std::uint16_t ELF_MACHINE_X86_64 = 62;
52 constexpr std::uint16_t ELF_MACHINE_AARCH64 = 183;
53 constexpr std::uint16_t ELF_MACHINE_RISCV = 243;
54
55 constexpr std::uint64_t FNV_OFFSET_BASIS = 14695981039346656037ULL;
56 constexpr std::uint64_t FNV_PRIME = 1099511628211ULL;
57 constexpr std::uint64_t HEX_FIELD_WIDTH_64 = 16;
58
59 bool has_range(std::span<const std::uint8_t> bytes, std::uint64_t offset, std::uint64_t size)
60 {
61     const std::uint64_t byte_count = static_cast<std::uint64_t>(bytes.size());
62     return offset <= byte_count && size <= byte_count - offset;
63 }
64
65 std::uint8_t read_u8(std::span<const std::uint8_t> bytes, std::uint64_t offset)
66 {
67     if (!has_range(bytes, offset, sizeof(std::uint8_t))) {
68         throw std::out_of_range("read_u8 outside input");
69     }
70     return bytes[static_cast<std::size_t>(offset)];
71 }
72
73 std::uint16_t read_u16_le(std::span<const std::uint8_t> bytes, std::uint64_t offset)
74 {
75     if (!has_range(bytes, offset, sizeof(std::uint16_t))) {
76         throw std::out_of_range("read_u16_le outside input");
77     }
78     const std::uint16_t low = bytes[static_cast<std::size_t>(offset)];
79     const std::uint16_t high = bytes[static_cast<std::size_t>(offset + 1)];
80     return static_cast<std::uint16_t>(low | static_cast<std::uint16_t>(high << 8U));
81 }
82
83 std::uint32_t read_u32_le(std::span<const std::uint8_t> bytes, std::uint64_t offset)
84 {
85     if (!has_range(bytes, offset, sizeof(std::uint32_t))) {
86         throw std::out_of_range("read_u32_le outside input");
87     }

```

```

88     std::uint32_t value = 0;
89     constexpr std::uint32_t BYTE_BITS = 8;
90     for (std::uint32_t i = 0; i < sizeof(std::uint32_t); ++i) {
91         value |= static_cast<std::uint32_t>(bytes[static_cast<std::size_t>(↵
↵ offset + i)])
92             << (i * BYTE_BITS);
93     }
94     return value;
95 }
96
97 std::uint64_t read_u64_le(std::span<const std::uint8_t> bytes, std::uint64_t ↵
↵ offset)
98 {
99     if (!has_range(bytes, offset, sizeof(std::uint64_t))) {
100         throw std::out_of_range("read_u64_le outside input");
101     }
102     std::uint64_t value = 0;
103     constexpr std::uint64_t BYTE_BITS = 8;
104     for (std::uint64_t i = 0; i < sizeof(std::uint64_t); ++i) {
105         value |= static_cast<std::uint64_t>(bytes[static_cast<std::size_t>(↵
↵ offset + i)])
106             << (i * BYTE_BITS);
107     }
108     return value;
109 }
110
111 void write_hex64(std::ostream& output, std::uint64_t value)
112 {
113     output << "0x" << std::hex << std::setfill('0') << std::setw(↵
↵ HEX_FIELD_WIDTH_64) << value
114         << std::dec << std::setfill(' ');
115 }
116
117 std::string read_string(std::span<const std::uint8_t> table, std::uint32_t ↵
↵ offset)
118 {
119     if (offset >= table.size()) {
120         return {};
121     }
122
123     std::string value;
124     for (std::size_t i = offset; i < table.size() && table[i] != 0; ++i) {
125         value.push_back(static_cast<char>(table[i]));
126     }
127     return value;
128 }
129
130 std::span<const std::uint8_t> slice_bytes(
131     std::span<const std::uint8_t> bytes,
132     std::uint64_t offset,
133     std::uint64_t size)
134 {

```

```

135     if (!has_range(bytes, offset, size)) {
136         return {};
137     }
138     return bytes.subspan(static_cast<std::size_t>(offset), static_cast<std::size_t>(size));
139 }
140
141 struct RawSection {
142     std::uint32_t name_offset{};
143     std::uint32_t type{};
144     std::uint64_t flags{};
145     std::uint64_t address{};
146     std::uint64_t offset{};
147     std::uint64_t size{};
148     std::uint32_t link{};
149     std::uint64_t entry_size{};
150 };
151
152 RawSection read_raw_section(std::span<const std::uint8_t> bytes, std::uint64_t section_offset)
153 {
154     return RawSection{
155         .name_offset = read_u32_le(bytes, section_offset + ELF64_SECTION_NAME_OFFSET),
156         .type = read_u32_le(bytes, section_offset + ELF64_SECTION_TYPE_OFFSET),
157         .flags = read_u64_le(bytes, section_offset + ELF64_SECTION_FLAGS_OFFSET),
158         .address = read_u64_le(bytes, section_offset + ELF64_SECTION_ADDRESS_OFFSET),
159         .offset = read_u64_le(bytes, section_offset + ELF64_SECTION_FILE_OFFSET),
160         .size = read_u64_le(bytes, section_offset + ELF64_SECTION_SIZE_OFFSET),
161         .link = read_u32_le(bytes, section_offset + ELF64_SECTION_LINK_OFFSET),
162         .entry_size = read_u64_le(bytes, section_offset + ELF64_SECTION_ENTRY_SIZE_OFFSET),
163     };
164 }
165
166 std::vector<RawSection> read_raw_sections(
167     std::span<const std::uint8_t> bytes,
168     std::uint64_t section_header_offset,
169     std::uint16_t section_count,
170     std::uint16_t section_entry_size,
171     std::vector<std::string>& diagnostics)
172 {
173     std::vector<RawSection> sections;
174     if (section_entry_size < ELF64_SECTION_HEADER_SIZE) {
175         diagnostics.push_back("section entry size is smaller than ELF64 section header");
176         return sections;
177     }
178 }

```

```

179 sections.reserve(section_count);
180 for (std::uint16_t index = 0; index < section_count; ++index) {
181     const std::uint64_t current_offset =
182         section_header_offset + static_cast<std::uint64_t>(index) * ↵
↵ section_entry_size;
183     if (!has_range(bytes, current_offset, ELF64_SECTION_HEADER_SIZE)) {
184         diagnostics.push_back("section header table extends beyond input");
185         break;
186     }
187     sections.push_back(read_raw_section(bytes, current_offset));
188 }
189 return sections;
190 }
191
192 std::vector<SectionSummary> summarize_sections(
193     std::span<const std::uint8_t> bytes,
194     const std::vector<RawSection>& raw_sections,
195     std::uint16_t section_name_index,
196     std::vector<std::string>& diagnostics)
197 {
198     if (section_name_index >= raw_sections.size()) {
199         diagnostics.push_back("section-name string table index is out of range"↵
↵ );
200     }
201     return {};
202 }
203
204 const RawSection& name_section = raw_sections[section_name_index];
205 const std::span<const std::uint8_t> names =
206     slice_bytes(bytes, name_section.offset, name_section.size);
207 if (names.empty() && name_section.size != 0) {
208     diagnostics.push_back("section-name string table extends beyond input")↵
↵ ;
209 }
210 return {};
211
212 std::vector<SectionSummary> summaries;
213 summaries.reserve(raw_sections.size());
214 for (const RawSection& raw : raw_sections) {
215     summaries.push_back(SectionSummary{
216         .name = read_string(names, raw.name_offset),
217         .type = raw.type,
218         .flags = raw.flags,
219         .address = raw.address,
220         .offset = raw.offset,
221         .size = raw.size,
222     });
223 }
224 return summaries;
225 }
226
227 std::vector<SymbolSummary> summarize_symbols(
228     std::span<const std::uint8_t> bytes,

```

```

228     const std::vector<RawSection>& raw_sections,
229     const std::vector<SectionSummary>& sections,
230     const InspectionConfig& config,
231     std::vector<std::string>& diagnostics)
232 {
233     std::vector<SymbolSummary> symbols;
234     if (!config.include_symbols) {
235         return symbols;
236     }
237
238     for (std::size_t section_index = 0; section_index < raw_sections.size(); ++↵
↵ section_index) {
239         const RawSection& symbol_section = raw_sections[section_index];
240         if (symbol_section.type != ELF_SECTION_TYPE_SYMTAB &&
241             symbol_section.type != ELF_SECTION_TYPE_DYNSYM) {
242             continue;
243         }
244         if (symbol_section.entry_size < ELF64_SYMBOL_ENTRY_SIZE) {
245             diagnostics.push_back("symbol entry size is smaller than ELF64 ↵
↵ symbol entry");
246             continue;
247         }
248         if (symbol_section.link >= raw_sections.size()) {
249             diagnostics.push_back("symbol string table link is out of range");
250             continue;
251         }
252
253         const RawSection& string_section = raw_sections[symbol_section.link];
254         const std::span<const std::uint8_t> symbol_names =
255             slice_bytes(bytes, string_section.offset, string_section.size);
256         const std::uint64_t symbol_count = symbol_section.size / symbol_section↵
↵ .entry_size;
257         const std::uint64_t capped_count = std::min(symbol_count, config.↵
↵ max_symbols);
258
259         for (std::uint64_t symbol_index = 0; symbol_index < capped_count; ++↵
↵ symbol_index) {
260             const std::uint64_t symbol_offset =
261                 symbol_section.offset + symbol_index * symbol_section.↵
↵ entry_size;
262             if (!has_range(bytes, symbol_offset, ELF64_SYMBOL_ENTRY_SIZE)) {
263                 diagnostics.push_back("symbol table extends beyond input");
264                 break;
265             }
266
267             const std::uint32_t name_offset =
268                 read_u32_le(bytes, symbol_offset + ELF64_SYMBOL_NAME_OFFSET);
269             const std::uint8_t info = read_u8(bytes, symbol_offset + ↵
↵ ELF64_SYMBOL_INFO_OFFSET);
270             const std::uint16_t symbol_section_index =
271                 read_u16_le(bytes, symbol_offset + ↵
↵ ELF64_SYMBOL_SECTION_INDEX_OFFSET);

```

```

272         const std::string section_name =
273             symbol_section_index < sections.size() ? sections[
↳ symbol_section_index].name : "";
274
275         symbols.push_back(SymbolSummary{
276             .name = read_string(symbol_names, name_offset),
277             .section_name = section_name,
278             .type = static_cast<std::uint8_t>(info & 0x0FU),
279             .binding = static_cast<std::uint8_t>(info >> 4U),
280             .section_index = symbol_section_index,
281             .value = read_u64_le(bytes, symbol_offset + 
↳ ELF64_SYMBOL_VALUE_OFFSET),
282             .size = read_u64_le(bytes, symbol_offset + 
↳ ELF64_SYMBOL_SIZE_OFFSET),
283         });
284     }
285
286     if (symbol_count > config.max_symbols) {
287         diagnostics.push_back("symbol table truncated by max_symbols");
288     }
289 }
290
291 return symbols;
292 }
293
294 bool has_elf_magic(std::span<const std::uint8_t> bytes)
295 {
296     return bytes.size() >= ELF_IDENT_SIZE && bytes[0] == ELF_MAGIC_0 && bytes[
↳ 1] == ELF_MAGIC_1 &&
↳ bytes[2] == ELF_MAGIC_2 && bytes[3] == ELF_MAGIC_3;
298 }
299
300 } // namespace
301
302 std::vector<std::uint8_t> read_binary_file(const std::filesystem::path& path)
303 {
304     std::ifstream input(path, std::ios::binary);
305     if (!input) {
306         throw std::runtime_error("failed to open binary file: " + path.string()
↳ );
307     }
308
309     input.seekg(0, std::ios::end);
310     const std::streamoff size = input.tellg();
311     if (size < 0) {
312         throw std::runtime_error("failed to determine binary file size: " + 
↳ path.string());
313     }
314     input.seekg(0, std::ios::beg);
315
316     std::vector<std::uint8_t> bytes(static_cast<std::size_t>(size));
317     if (!bytes.empty()) {

```

```

318     input.read(reinterpret_cast<char*>(bytes.data()), size);
319 }
320 if (!input && !input.eof()) {
321     throw std::runtime_error("failed to read binary file: " + path.string()↵
↵ );
322 }
323 return bytes;
324 }
325
326 std::uint64_t checksum_bytes(std::span<const std::uint8_t> bytes)
327 {
328     std::uint64_t value = FNV_OFFSET_BASIS;
329     for (const std::uint8_t byte : bytes) {
330         value ^= byte;
331         value *= FNV_PRIME;
332     }
333     return value;
334 }
335
336 std::string classify_machine(std::uint16_t machine)
337 {
338     switch (machine) {
339     case ELF_MACHINE_X86_64:
340         return "x86-64";
341     case ELF_MACHINE_AARCH64:
342         return "AArch64";
343     case ELF_MACHINE_RISCV:
344         return "RISC-V";
345     default:
346         return "unknown";
347     }
348 }
349
350 ElfReport inspect_elf(
351     std::span<const std::uint8_t> bytes,
352     std::string_view source_name,
353     const InspectionConfig& config)
354 {
355     ElfReport report;
356     report.source_name = std::string(source_name);
357     report.byte_size = bytes.size();
358     report.checksum = checksum_bytes(bytes);
359     report.valid_magic = has_elf_magic(bytes);
360
361     if (!report.valid_magic) {
362         report.diagnostics.push_back("input does not start with ELF magic");
363         return report;
364     }
365     if (bytes.size() < ELF64_HEADER_MIN_SIZE) {
366         report.diagnostics.push_back("input is shorter than ELF64 header");
367         return report;
368     }

```

```

369
370     report.is_elf64 = read_u8(bytes, ELF_IDENT_CLASS_OFFSET) == ELF_CLASS_64;
371     report.is_little_endian = read_u8(bytes, ELF_IDENT_DATA_OFFSET) == ↵
↵ ELF_DATA_LITTLE_ENDIAN;
372     if (!report.is_elf64 || !report.is_little_endian) {
373         report.diagnostics.push_back("only ELF64 little-endian inputs are ↵
↵ supported");
374         return report;
375     }
376
377     report.object_type = read_u16_le(bytes, ELF64_TYPE_OFFSET);
378     report.machine = read_u16_le(bytes, ELF64_MACHINE_OFFSET);
379     report.entry_address = read_u64_le(bytes, ELF64_ENTRY_OFFSET);
380
381     const std::uint64_t section_header_offset = read_u64_le(bytes, ↵
↵ ELF64_SECTION_HEADER_OFFSET);
382     const std::uint16_t section_entry_size =
383         read_u16_le(bytes, ELF64_HEADER_SECTION_ENTRY_SIZE_OFFSET);
384     const std::uint16_t section_count = read_u16_le(bytes, ↵
↵ ELF64_SECTION_COUNT_OFFSET);
385     const std::uint16_t section_name_index = read_u16_le(bytes, ↵
↵ ELF64_SECTION_NAME_INDEX_OFFSET);
386
387     std::vector<RawSection> raw_sections = read_raw_sections(
388         bytes,
389         section_header_offset,
390         section_count,
391         section_entry_size,
392         report.diagnostics);
393     report.sections = summarize_sections(bytes, raw_sections, ↵
↵ section_name_index, report.diagnostics);
394     report.symbols =
395         summarize_symbols(bytes, raw_sections, report.sections, config, report.↵
↵ diagnostics);
396
397     return report;
398 }
399
400 std::string format_report(const ElfReport& report)
401 {
402     std::ostringstream output;
403     output << "source=" << report.source_name << '\n';
404     output << "bytes=" << report.byte_size << '\n';
405     output << "checksum=" << report.checksum << '\n';
406     output << "valid_magic=" << (report.valid_magic ? "true" : "false") << '\n'↵
↵ ;
407     output << "class=" << (report.is_elf64 ? "ELF64" : "unsupported") << '\n';
408     output << "endian=" << (report.is_little_endian ? "little" : "unsupported")↵
↵ << '\n';
409     output << "machine=" << classify_machine(report.machine) << '\n';
410     output << "entry=";
411     write_hex64(output, report.entry_address);

```

```

412     output << '\n';
413     output << "sections=" << report.sections.size() << '\n';
414     output << "symbols=" << report.symbols.size() << '\n';
415     for (const std::string& diagnostic : report.diagnostics) {
416         output << "diagnostic=" << diagnostic << '\n';
417     }
418     return output.str();
419 }
420
421 std::string format_sections_csv(const ElfReport& report)
422 {
423     std::ostringstream output;
424     output << "name,type,flags,address,offset,size\n";
425     for (const SectionSummary& section : report.sections) {
426         output << section.name << ',' << section.type << ',' << section.flags <
↵ << ','
427         << section.address << ',' << section.offset << ',' << section.↵
↵ size << '\n';
428     }
429     return output.str();
430 }
431
432 } // namespace cpu1

```

Listing 2.7 src/main.cpp

```

1  #include <cpu1/executable_evidence.hpp>
2
3  #include <exception>
4  #include <iostream>
5  #include <string_view>
6
7  namespace {
8
9  constexpr int CLI_SUCCESS = 0;
10 constexpr int CLI_USAGE_ERROR = 2;
11 constexpr int CLI_RUNTIME_ERROR = 1;
12 constexpr int CLI_MIN_ARGUMENTS = 2;
13 constexpr int CLI_MODE_ARGUMENTS = 3;
14
15 void print_usage(std::ostream& output)
16 {
17     output << "usage: cpu1ee_cli <elf-file> [--sections-csv]\n";
18 }
19
20 } // namespace
21
22 int main(int argc, char** argv)
23 {
24     if (argc < CLI_MIN_ARGUMENTS || argc > CLI_MODE_ARGUMENTS) {
25         print_usage(std::cerr);
26         return CLI_USAGE_ERROR;

```

```

27     }
28
29     try {
30         const std::vector<std::uint8_t> bytes = cpu1::read_binary_file(argv[1])↵
↵ ;
31         const cpu1::ElfReport report = cpu1::inspect_elf(bytes, argv[1]);
32
33         if (argc == CLI_MODE_ARGUMENTS && std::string_view(argv[2]) == "--↵
↵ sections-csv") {
34             std::cout << cpu1::format_sections_csv(report);
35             return CLI_SUCCESS;
36         }
37
38         std::cout << cpu1::format_report(report);
39         return CLI_SUCCESS;
40     } catch (const std::exception& error) {
41         std::cerr << "cpu1ee_cli: " << error.what() << '\n';
42         return CLI_RUNTIME_ERROR;
43     }
44 }

```

四、测试和 Benchmark

Listing 2.8 tests/executable_evidence_tests.cpp

```

1  #include <cpu1/elf_fixture.hpp>
2  #include <cpu1/executable_evidence.hpp>
3
4  #include <gmock/gmock.h>
5  #include <gtest/gtest.h>
6
7  #include <cstdint>
8  #include <filesystem>
9  #include <fstream>
10 #include <string>
11 #include <vector>
12
13 namespace {
14
15 using ::testing::HasSubstr;
16
17 constexpr std::uint16_t TEST_ELF_MACHINE_AARCH64 = 183;
18 constexpr std::uint16_t TEST_ELF_MACHINE_RISCV = 243;
19 constexpr std::uint16_t TEST_ELF_MACHINE_UNKNOWN = 0;
20
21 } // namespace
22
23 TEST(CPU1ExecutableEvidenceTest, ParsesSyntheticElfHeaderSectionsAndSymbols)
24 {
25     const std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
26     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "fixture.elf");

```

```

27
28 EXPECT_TRUE(report.valid_magic);
29 EXPECT_TRUE(report.is_elf64);
30 EXPECT_TRUE(report.is_little_endian);
31 EXPECT_EQ(report.object_type, cpu1::FIXTURE_ELF_EXEC_TYPE);
32 EXPECT_EQ(report.machine, cpu1::FIXTURE_ELF_X86_64);
33 EXPECT_EQ(report.entry_address, cpu1::FIXTURE_ELF_ENTRY);
34 EXPECT_EQ(report.byte_size, cpu1::FIXTURE_ELF_SIZE);
35 EXPECT_TRUE(report.diagnostics.empty());
36 ASSERT_EQ(report.sections.size(), cpu1::FIXTURE_ELF_SH_COUNT);
37 EXPECT_EQ(report.sections[1].name, ".text");
38 EXPECT_EQ(report.sections[1].offset, cpu1::FIXTURE_ELF_TEXT_OFFSET);
39 EXPECT_EQ(report.sections[1].size, cpu1::FIXTURE_ELF_TEXT_SIZE);
40 ASSERT_EQ(report.symbols.size(), 3U);
41 EXPECT_EQ(report.symbols[1].name, "_start");
42 EXPECT_EQ(report.symbols[1].section_name, ".text");
43 EXPECT_EQ(report.symbols[1].binding, 1U);
44 EXPECT_EQ(report.symbols[1].type, 2U);
45 }
46
47 TEST(CPU1ExecutableEvidenceTest, FormatsStableReportAndSectionCsv)
48 {
49     const std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
50     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "fixture.elf");
51
52     const std::string text = cpu1::format_report(report);
53     const std::string csv = cpu1::format_sections_csv(report);
54
55     EXPECT_THAT(text, HasSubstr("source=fixture.elf"));
56     EXPECT_THAT(text, HasSubstr("machine=x86-64"));
57     EXPECT_THAT(text, HasSubstr("sections=5"));
58     EXPECT_THAT(text, HasSubstr("symbols=3"));
59     EXPECT_THAT(csv, HasSubstr("name,type,flags,address,offset,size"));
60     EXPECT_THAT(csv, HasSubstr(".text,1,6,4198400,96,4"));
61 }
62
63 TEST(CPU1ExecutableEvidenceTest, RejectsNonElfMagic)
64 {
65     const std::vector<std::uint8_t> bytes{0, 1, 2, 3};
66     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "bad.bin");
67
68     EXPECT_FALSE(report.valid_magic);
69     ASSERT_EQ(report.diagnostics.size(), 1U);
70     EXPECT_EQ(report.diagnostics[0], "input does not start with ELF magic");
71 }
72
73 TEST(CPU1ExecutableEvidenceTest, RejectsShortElfHeader)
74 {
75     std::vector<std::uint8_t> bytes(cpu1::FIXTURE_ELF_IDENT_SIZE, 0);
76     bytes[0] = cpu1::FIXTURE_ELF_MAGIC_0;
77     bytes[1] = cpu1::FIXTURE_ELF_MAGIC_1;
78     bytes[2] = cpu1::FIXTURE_ELF_MAGIC_2;

```

```

79     bytes[3] = cpu1::FIXTURE_ELF_MAGIC_3;
80
81     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "short.elf");
82
83     EXPECT_TRUE(report.valid_magic);
84     ASSERT_EQ(report.diagnostics.size(), 1U);
85     EXPECT_EQ(report.diagnostics[0], "input is shorter than ELF64 header");
86 }
87
88 TEST(CPU1ExecutableEvidenceTest, RejectsUnsupportedClassOrEndian)
89 {
90     std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
91     bytes[4] = 1;
92
93     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "elf32.elf");
94
95     EXPECT_TRUE(report.valid_magic);
96     EXPECT_FALSE(report.is_elf64);
97     ASSERT_EQ(report.diagnostics.size(), 1U);
98     EXPECT_EQ(report.diagnostics[0], "only ELF64 little-endian inputs are ←
    ↪ supported");
99 }
100
101 TEST(CPU1ExecutableEvidenceTest, RespectsSymbolCap)
102 {
103     const std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
104     cpu1::InspectionConfig config;
105     config.max_symbols = 1;
106
107     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "fixture.elf", ←
    ↪ config);
108
109     EXPECT_EQ(report.symbols.size(), 1U);
110     ASSERT_EQ(report.diagnostics.size(), 1U);
111     EXPECT_EQ(report.diagnostics[0], "symbol table truncated by max_symbols");
112 }
113
114 TEST(CPU1ExecutableEvidenceTest, ClassifiesKnownAndUnknownMachines)
115 {
116     EXPECT_EQ(cpu1::classify_machine(cpu1::FIXTURE_ELF_X86_64), "x86-64");
117     EXPECT_EQ(cpu1::classify_machine(TEST_ELF_MACHINE_AARCH64), "AArch64");
118     EXPECT_EQ(cpu1::classify_machine(TEST_ELF_MACHINE_RISCV), "RISC-V");
119     EXPECT_EQ(cpu1::classify_machine(TEST_ELF_MACHINE_UNKNOWN), "unknown");
120 }
121
122 TEST(CPU1ExecutableEvidenceTest, ReadsBinaryFileAndKeepsChecksumStable)
123 {
124     const std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
125     const std::filesystem::path path =
126         std::filesystem::temp_directory_path() / "cpu1ee_fixture.elf";
127     {
128         std::ofstream output(path, std::ios::binary);

```

```
129     ASSERT_TRUE(output);
130     output.write(reinterpret_cast<const char*>(bytes.data()),
131                  static_cast<std::streamsize>(bytes.size()));
132 }
133
134 const std::vector<std::uint8_t> read_back = cpu1::read_binary_file(path);
135 std::filesystem::remove(path);
136
137 EXPECT_EQ(read_back, bytes);
138 EXPECT_EQ(cpu1::checksum_bytes(read_back), cpu1::checksum_bytes(bytes));
139 }
```

Listing 2.9 bench/executable_evidence_bench.cpp

```
1 #include <cpu1/elf_fixture.hpp>
2 #include <cpu1/executable_evidence.hpp>
3
4 #include <benchmark/benchmark.h>
5
6 #include <cstdint>
7 #include <vector>
8
9 static void BMInspectSyntheticElf(benchmark::State& state)
10 {
11     const std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
12     for (auto _ : state) {
13         const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "bench.elf");
14         benchmark::DoNotOptimize(report.sections.size());
15         benchmark::DoNotOptimize(report.symbols.size());
16         benchmark::ClobberMemory();
17     }
18 }
19
20 BENCHMARK(BMInspectSyntheticElf);
21 BENCHMARK_MAIN();
```

术语表

可执行文件证据 从二进制文件、工具输出、测试和报告中得到的可复查事实，例如 checksum、machine、entry、section 和 symbol。

ELF header ELF 文件开头的固定结构，保存 class、endianness、machine、entry 和 section/program header 位置等信息。

section header table section 目录表，记录每个 section 的名字偏移、类型、flags、地址、文件偏移和大小。

string table 字符串表。ELF 结构里很多名字通过偏移引用它，而不是直接保存字符串。

symbol table 符号表，把函数或对象名字、绑定、类型、section index、地址和大小连接起来。

diagnostic 诊断信息。解析器遇到不支持或不安全的输入时，用 diagnostic 说明原因，而不是继续读垃圾。

benchmark smoke 小型 benchmark 检查。它证明 benchmark 入口能跑，但不等于完整性能结论。

交叉验证 用两个独立来源检查同一事实，例如用本卷解析器和 readelf 同时验证 machine 或 section size。

索引

Executable Evidence Kit, ii